

DOCUMENTATION TECHNIQUE

Application de gestion de notes organisées par espaces

Architecture client-serveur en Go

Contexte — Test technique, poste de mentor
Filière Informatique & Cybersécurité

Auteur — Youssef Ammari

Dépôt serveur — `test-technique-ynov`

Dépôt client — `test-technique-ynov-client`

Sommaire

1. Présentation et périmètre

1. Le besoin fonctionnel
2. Couverture des fonctionnalités demandées

2. Choix techniques

1. Tableau de synthèse
2. Justification des choix structurants
3. Ce qui a été volontairement écarté

3. Architecture globale

1. Vue d'ensemble
2. Flux d'une requête authentifiée
3. Répartition des responsabilités

4. Organisation du code

1. Dépôt serveur
2. Dépôt client
3. Règle de dépendance

5. Modélisation des données

1. Entités et relations
2. Schéma relationnel
3. Invariants garantis par la base

6. Partis pris d'implémentation

1. Authentification
2. Contrôle d'accès aux données
3. Gestion des erreurs
4. Validation des données
5. Persistance
6. Gestion des états de note
7. Contraintes propres au client HTML

7. Stratégie de tests

8. Méthode de travail et versionnement

9. Limites, simplifications et compromis

10. Mise en route

1. Présentation et périmètre

1.1 Le besoin fonctionnel

L'application permet à un utilisateur authentifié d'organiser ses notes dans des **espaces** thématiques : « Devoirs », « Jobs », « Personnel »... Un espace doit exister avant qu'une note puisse y être ajoutée, et chaque note appartient obligatoirement à un espace.

La règle transversale, qui structure l'ensemble de la solution, est la suivante : **un utilisateur ne doit jamais pouvoir consulter ni modifier les espaces et les notes d'un autre utilisateur**. Cette contrainte n'est pas traitée comme une vérification ajoutée après coup, mais comme une propriété de la conception — la section 6.2 détaille comment.

1.2 Couverture des fonctionnalités demandées

Réf.	Fonctionnalité	Réalisation
FT1	Authentification	Inscription et connexion par email et mot de passe, jeton JWT, comptes de démonstration fournis
FT2	Gestion des espaces	Création, liste, consultation, modification, suppression
FT3	Consultation des notes d'un espace	Notes affichées par espace, avec leur état
FT4	Création d'une note	Depuis un espace existant, rattachement automatique
FT5	Modification d'une note	Titre, contenu et état modifiables
FT6	Suppression d'une note	Réservée au propriétaire de l'espace

Sur l'intitulé de FT6

L'énoncé titre cette fonctionnalité « Une affectation des tâches », mais son corps décrit la suppression d'une note par son propriétaire. C'est ce contenu qui a été implémenté. Aucune notion d'affectation à un tiers n'a été ajoutée : le modèle de données du sujet ne comporte qu'un seul utilisateur par espace, et l'introduire aurait supposé une notion de partage absente de l'énoncé.

2. Choix techniques

2.1 Tableau de synthèse

Besoin	Choix	Raison
Langage (serveur et client)	Go 1.25	Imposé par le sujet
Routeur HTTP du serveur	Gin 1.12	Groupes de routes et middlewares concis, large adoption, documentation abondante
Routeur HTTP du client	net/http (standard)	Depuis Go 1.22 la bibliothèque standard gère méthode et paramètres d'URL ; une quinzaine de routes ne justifie aucune dépendance
Base de données	PostgreSQL 16	Base relationnelle imposée ; clés étrangères, contraintes CHECK et index partiels natifs
Accès aux données	database/sql + pilote pgx	SQL écrit à la main : chaque requête est lisible et explicable, le contrôle d'accès est visible dans la clause WHERE
Hachage des mots de passe	bcrypt	Fonction lente et salée, conçue pour les mots de passe
Jetons d'authentification	JWT HS256 (golang-jwt/jwt/v5)	Imposé par le sujet ; API sans état
Validation des entrées	go-playground/validator (via Gin)	Déjà présent dans Gin ; règles déclarées au plus près des structures de requête
Gabarits HTML	html/template	Imposé par le sujet ; échappement contextuel automatique
Style	CSS écrit à la main	Le sujet porte sur Go, pas sur le front
Environnement de développement	Docker Compose	Base de données jetable et reproductible en une commande

2.2 Justification des choix structurants

SQL écrit à la main plutôt qu'un ORM

C'est le choix le plus déterminant du projet. Un ORM comme GORM aurait réduit le volume de code, mais le contrôle d'accès est ici la préoccupation centrale, et il s'exprime entièrement en SQL. Écrire les requêtes à la main rend visible, à la lecture d'une seule ligne, le fait qu'un utilisateur ne peut atteindre que ses propres données :

```
SELECT ... FROM spaces WHERE id = $1 AND user_id = $2
```

Avec un ORM, cette garantie serait diluée dans des appels chaînés dont le SQL généré n'est pas sous les yeux. Le coût assumé est un code plus verbeux : chaque `Scan` énumère ses colonnes.

Gin sur le serveur, bibliothèque standard sur le client

Les deux applications n'ont pas les mêmes besoins. Le serveur expose une API avec des groupes de routes, un middleware d'authentification appliqué à un groupe entier et un décodage JSON validé : Gin apporte là une réelle économie de code. Le client, lui, rend des gabarits et effectue des appels HTTP ; son routage tient en quinze lignes que la bibliothèque standard couvre depuis Go 1.22.

Utiliser une dépendance là où elle sert, et s'en passer là où elle n'apporte rien, est un choix délibéré plutôt qu'une incohérence. Le dépôt du client ne déclare d'ailleurs **aucune dépendance externe**.

Migrations SQL jouées par PostgreSQL lui-même

Les fichiers du dossier `migrations/` sont montés dans `/docker-entrypoint-initdb.d` et exécutés par PostgreSQL au premier démarrage du conteneur. Aucun outil de migration supplémentaire n'est à installer ni à apprendre pour faire tourner le projet. La limite de ce choix est traitée en section 9.

2.3 Ce qui a été volontairement écarté

Écarté	Pourquoi
Framework front (React, Vue)	Explicitement hors attendus du sujet
ORM	Masquerait le SQL qui porte le contrôle d'accès (section 2.2)
Conteneur d'injection de dépendances	L'assemblage manuel dans <code>main.go</code> tient en vingt lignes et se lit de haut en bas
Bibliothèque d'assertions de test (testify)	La bibliothèque standard suffit ; une dépendance de moins à justifier
Interfaces devant les repositories	Il n'existe qu'une implémentation. Les tests s'exécutent contre une vraie base, car c'est le comportement du SQL que l'on veut vérifier (section 7)
Jeton de rafraîchissement	Hors périmètre du sujet ; documenté comme limite en section 9

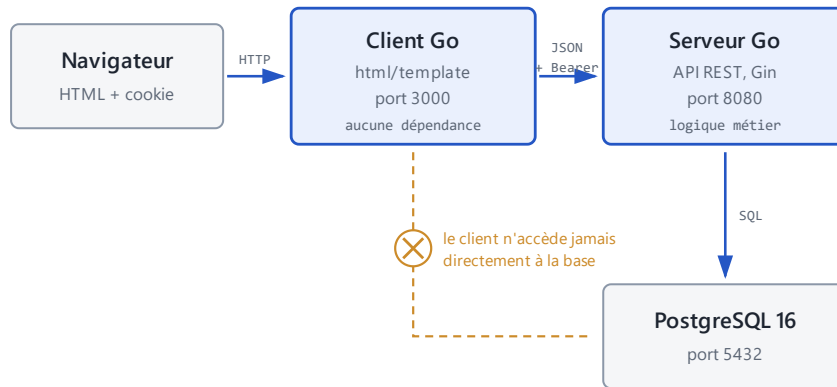
Sur l'absence d'interfaces

Introduire une interface `SpaceRepository` pour une unique implémentation aurait ajouté une indirection sans bénéfice. L'abstraction devient utile lorsqu'elle sert un besoin réel — plusieurs implémentations, ou un test qui ne peut pas atteindre la dépendance réelle. Ce n'est le cas ni de l'une ni de l'autre ici.

3. Architecture globale

3.1 Vue d'ensemble

La solution est composée de deux applications Go indépendantes, dans deux dépôts distincts, plus une base de données.



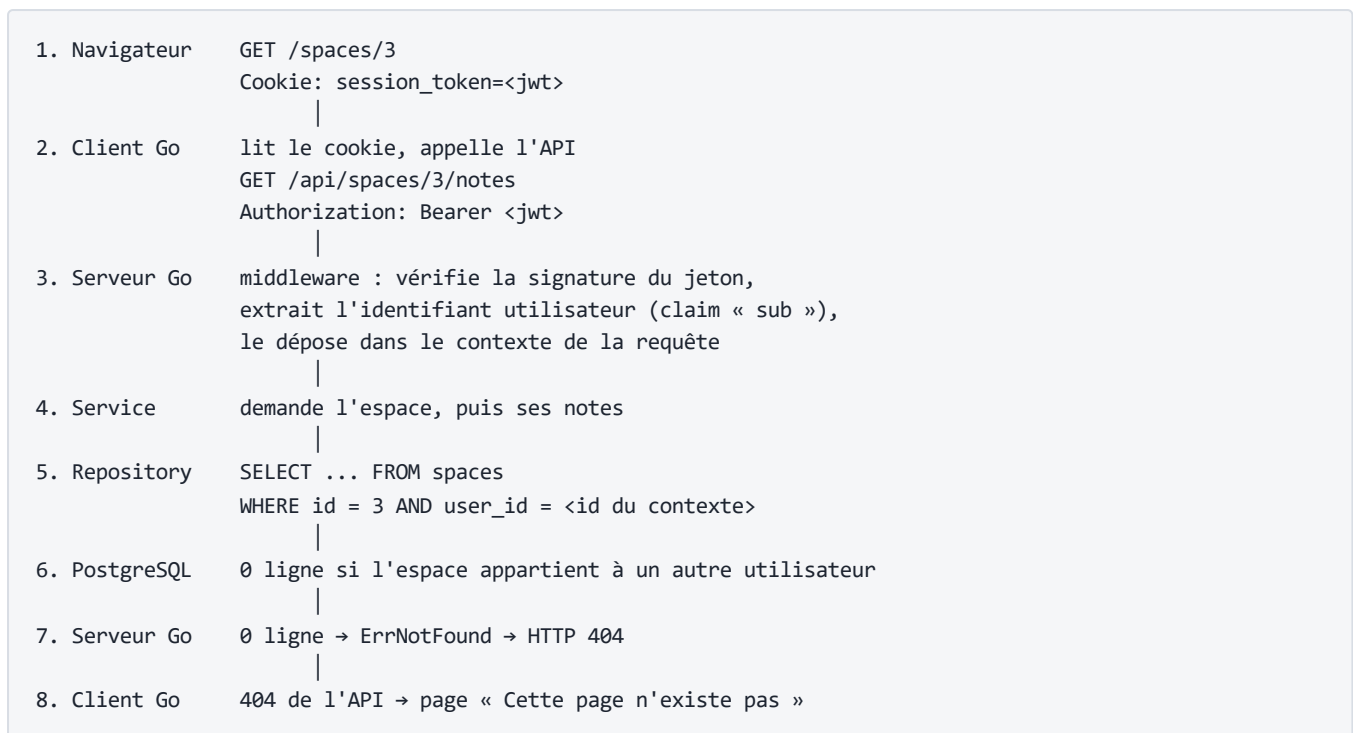
Architecture en trois couches et frontière d'accès aux données

Trois propriétés découlent de ce découpage :

- Le client **n'accède jamais à la base de données**. Il ne connaît que l'adresse de l'API.
- Le client **ne connaît pas le secret de signature des jetons**. Il reçoit un jeton, le stocke, le rejoue — il ne le vérifie pas.
- Le contrôle d'accès **n'est pas réimplémenté côté client**. Celui-ci affiche ce que l'API lui retourne. La sécurité ne dépend donc pas du client, qui pourrait être remplacé par une application mobile sans qu'aucune garantie ne soit perdue.

3.2 Flux d'une requête authentifiée

Exemple : l'utilisateur ouvre la page d'un espace.



Le point clé est à l'étape 5 : l'identifiant de l'utilisateur provient du **jeton vérifié**, jamais de l'URL ni du corps de la requête. Un client ne peut donc pas désigner l'utilisateur pour lequel il prétend agir.

3.3 Répartition des responsabilités

Responsabilité	Porté par	Commentaire
Logique métier	Serveur	Normalisation des données, règles d'accès
Accès aux données	Serveur	Seule couche à écrire du SQL
Authentification	Serveur	Hachage, signature et vérification des jetons
Validation des entrées	Serveur	Le client valide aussi, par confort ; seul le serveur fait foi
Contrôle d'accès	Serveur	Dans les clauses <code>WHERE</code> (section 6.2)
Rendu des pages	Client	<code>html/template</code> , échappement contextuel
Session navigateur	Client	Cookie <code>HttpOnly</code> portant le jeton
Navigaton et messages	Client	Redirections, confirmations, pages d'erreur

4. Organisation du code

4.1 Dépôt serveur

main.go	Point d'entrée : assemblage des briques, démarrage, arrêt propre. Aucune logique métier.
internal/	
apperrors/	Erreurs métier, indépendantes de HTTP
config/	Configuration lue dans l'environnement
database/	Ouverture et réglage du pool PostgreSQL
models/	Entités : User, Space, Note
repository/	Accès aux données, SQL écrit à la main
service/	Logique métier, orchestration
handlers/	Handlers HTTP, DTO, validation, format des erreurs
middleware/	Authentification par jeton
migrations/	Schéma SQL et données de démonstration
docs/	Cette documentation

4.2 Dépôt client

cmd/web/main.go	Point d'entrée : assemblage, démarrage, arrêt propre
internal/	
api/	Client HTTP typé : seul à connaître les routes de l'API
config/	Configuration lue dans l'environnement
handlers/	Handlers de pages, table des routes, session requise
render/	Compilation et exécution des gabarits
session/	Cookie portant le jeton JWT
web/	
templates/	layout, partials, pages
static/	Feuille de style
embed.go	Embarque templates/ et static/ dans le binaire

Le binaire du client est **autonome** : gabarits et CSS sont embarqués avec `go:embed`. Il fonctionne sans que le dossier `web/` soit déployé à côté de l'exécutable.

4.3 Règle de dépendance

Les deux applications suivent une dépendance à sens unique, sans retour :

```
Serveur : handlers → service → repository → PostgreSQL
Client  : handlers → api    → (HTTP)   → Serveur
```

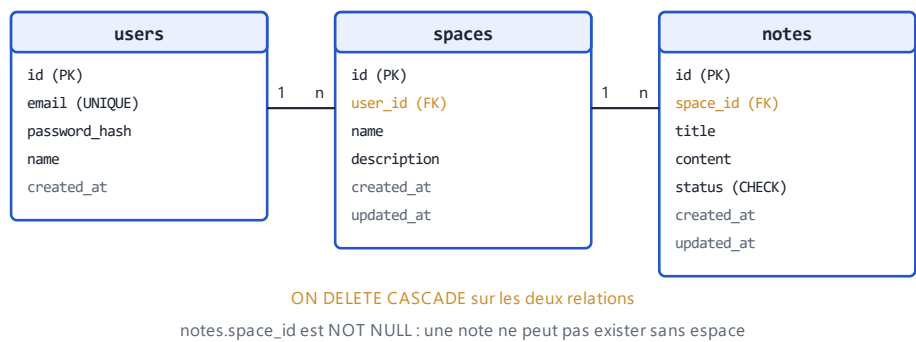
Chaque couche ne connaît que la suivante. Concrètement :

- Le `repository` ne sait pas ce qu'est un code de statut HTTP.
- Le `service` ne sait pas écrire de SQL.
- Les `handlers` ne contiennent aucune règle métier : ils décodent, appellent, sérialisent.

Cette discipline a une conséquence pratique : le remplacement de PostgreSQL ou le passage à un client mobile n'affecte qu'une seule couche.

5. Modélisation des données

5.1 Entités et relations



Modèle relationnel : un utilisateur possède des espaces, un espace contient des notes

5.2 Schéma relationnel

Les identifiants sont des `BIGSERIAL` . Les horodatages sont des `TIMESTAMPTZ` , donc porteurs de leur fuseau : une date reste correcte quel que soit le fuseau du serveur qui la relit.

Contrainte	Porte sur	Ce qu'elle garantit
<code>UNIQUE (email)</code>	<code>users</code>	Un email identifie un seul compte. L'application stocke l'email en minuscules, rendant la contrainte insensible à la casse
<code>UNIQUE (user_id, LOWER(name))</code>	<code>spaces</code>	Un utilisateur ne peut pas avoir deux espaces de même nom, mais deux utilisateurs peuvent chacun avoir un espace « Devoirs »
<code>NOT NULL + FK</code>	<code>notes.space_id</code>	Une note appartient obligatoirement à un espace existant
<code>CHECK (status IN ...)</code>	<code>notes.status</code>	Seuls <code>todo</code> , <code>in_progress</code> et <code>done</code> sont acceptés
<code>ON DELETE CASCADE</code>	les deux FK	Supprimer un espace supprime ses notes ; supprimer un utilisateur supprime tout son contenu. Aucune ligne orpheline possible
Index sur <code>spaces.user_id</code> et <code>notes.space_id</code>	—	Ces colonnes figurent dans toutes les clauses <code>WHERE</code> du contrôle d'accès

5.3 Invariants garantis par la base

Les règles ci-dessus sont posées **au niveau du schéma** et non uniquement dans le code Go. La distinction est importante : un invariant applicatif protège des erreurs d'usage, un invariant en base protège aussi des bugs applicatifs, des scripts de maintenance et des modifications manuelles.

Chacune a été vérifiée par une tentative de violation directe en SQL :

Tentative	Résultat
INSERT ... status = 'nawak'	Rejeté — notes_status_valid
INSERT espace « devoirs » alors que « Devoirs » existe	Rejeté — index unique insensible à la casse
INSERT note sans space_id	Rejeté — NOT NULL
DELETE d'un espace contenant 2 notes	Les 2 notes supprimées en cascade

6. Partis pris d'implémentation

6.1 Authentification

Stockage des mots de passe

Les mots de passe sont hachés avec **bcrypt** au coût par défaut. bcrypt est préféré à une fonction généraliste comme SHA-256 parce qu'il est volontairement lent et intègre un sel aléatoire : deux utilisateurs ayant choisi le même mot de passe ont des empreintes différentes, et les attaques par table précalculée deviennent impraticables. Le sel étant inclus dans la chaîne produite, il n'y a rien d'autre à stocker.

La structure `User` porte son empreinte dans un champ marqué `json:"-"` : le hachage est **structurellement insérialisable** dans une réponse de l'API. Ce n'est pas une omission à maintenir, c'est une impossibilité.

Les mots de passe sont en outre limités à 72 caractères à l'inscription. Cette limite n'est pas arbitraire : bcrypt ignore silencieusement les octets au-delà du 72^e. Accepter une phrase secrète plus longue laisserait croire à l'utilisateur qu'il est protégé par l'intégralité de sa saisie.

Jetons JWT

Le serveur signe des jetons HS256 dont la durée de vie est de 24 heures. L'identifiant de l'utilisateur est placé dans le champ standard `sub`, prévu par la spécification pour désigner le porteur.

La vérification impose explicitement l'algorithme attendu :

```
jwt.ParseWithClaims(token, &jwt.RegisteredClaims{}, keyFunc,  
    jwt.WithValidMethods([]string{jwt.SigningMethodHS256.Alg()}))
```

Pourquoi cette ligne est essentielle

Sans `WithValidMethods`, la bibliothèque fait confiance à l'algorithme déclaré *dans le jeton lui-même*. Un attaquant peut alors forger un jeton annonçant `alg: none`, sans signature, et se faire passer pour n'importe quel utilisateur. C'est la faille la plus répandue des implémentations JWT. Un test dédié (`TestParseRejectsUnsignedToken`) verrouille ce comportement, et la tentative a également été rejouée contre le serveur en fonctionnement.

Réponses aux échecs de connexion

Un email inconnu et un mot de passe erroné produisent **exactement la même réponse** : `401` et le message « email ou mot de passe incorrect ». Distinguer les deux cas permettrait de découvrir quelles adresses sont enregistrées, en soumettant simplement des emails au hasard.

6.2 Contrôle d'accès aux données

C'est le cœur du projet. Le parti pris tient en une phrase : **le contrôle d'accès est porté par la signature des méthodes du repository, pas par des vérifications que l'appelant doit penser à écrire.**

Toutes les méthodes exigent l'identifiant de l'utilisateur et l'intègrent à leur requête :

```
func (r *SpaceRepository) GetByID(ctx context.Context,  
    userID, spaceID int64) (*models.Space, error)
```

Il n'existe **aucune** méthode permettant de lire un espace sans fournir un `userID`. Un développeur distrait ne peut donc pas contourner le filtre : le code ne compilerait pas.

Le cas des notes

Les notes sont plus délicates : elles ne portent pas de colonne `user_id`. Leur propriétaire est celui de l'espace qui les contient. Chaque requête effectue donc une jointure plutôt que de faire confiance à un contrôle réalisé en amont :

```
SELECT n.* FROM notes n
JOIN spaces s ON s.id = n.space_id
WHERE n.id = $1 AND s.user_id = $2
```

Pour la création, l'approche naïve serait de lire l'espace, vérifier son propriétaire, puis insérer. Cela demande deux requêtes et laisse une fenêtre pendant laquelle l'espace peut disparaître. Une seule requête suffit :

```
INSERT INTO notes (space_id, title, content, status)
SELECT $1, $2, $3, $4
WHERE EXISTS (SELECT 1 FROM spaces WHERE id = $1 AND user_id = $5)
RETURNING id, space_id, title, content, status, created_at, updated_at
```

Si l'espace n'appartient pas à l'utilisateur, **aucune ligne n'est insérée**, `RETURNING` ne renvoie rien, et le repository traduit ce vide en `ErrNotFound`. L'opération est atomique.

404 plutôt que 403

Une ressource appartenant à autrui provoque un `404`, et non un `403`. Répondre `403` confirmerait que la ressource existe, permettant à un utilisateur de cartographier les données des autres en énumérant les identifiants. Les deux situations — « n'existe pas » et « ne vous appartient pas » — sont volontairement indistinguables de l'extérieur.

Vérification

Le cloisonnement a été éprouvé de deux manières. D'abord par des tests d'intégration contre une vraie base (`TestSpaceRepositoryIsolatesUsers`, `TestNoteRepositoryIsolatesUsers`), qui couvrent la lecture, la modification, la suppression, le listage et la création. Ensuite par des requêtes réelles contre le serveur puis contre le client, avec deux comptes :

Tentative de l'utilisateur B sur les données de A	Réponse
Lire un espace	404
Modifier un espace	404
Supprimer un espace	404
Lister les notes d'un espace	404
Créer une note dans un espace	404 — et 0 ligne insérée
Lire, modifier ou supprimer une note	404

Après chaque série de tentatives, les données de A ont été relues et trouvées intactes : aucune opération n'avait abouti même partiellement.

6.3 Gestion des erreurs

Côté serveur

Les couches basses retournent des **erreurs métier typées** (`ErrNotFound`, `ErrEmailAlreadyUsed`, `ErrInvalidCredentials` ...) définies dans le paquet `apperrors`, qui ne connaît pas HTTP. Une **unique fonction**, `respondError`, les traduit en codes de statut.

Cette centralisation a deux effets. Les réponses sont cohérentes sur toute l'API, et il n'existe qu'un seul endroit à relire pour savoir ce que renvoie le serveur dans chaque cas. Aucun `http.Error` n'est dispersé dans les handlers.

Statut	Cas
200	Succès
201	Ressource créée
204	Suppression effectuée, rien à retourner
400	Corps illisible, données invalides, identifiant d'URL non numérique
401	Jeton absent, invalide ou expiré ; identifiants incorrects
404	Ressource inexistante <i>ou</i> appartenant à un autre utilisateur
409	Email ou nom d'espace déjà utilisé
500	Erreur inattendue

Les erreurs inattendues sont **journalisées avec leur détail** côté serveur, mais le client ne reçoit qu'un message générique. Renvoyer l'erreur brute exposerait la structure de la base ou des informations d'infrastructure.

Les erreurs techniques de PostgreSQL sont traduites au plus près de leur source : le repository intercepte le code `23505` (violation d'unicité) et le convertit en `ErrEmailAlreadyUsed` ou `ErrSpaceNameAlreadyUsed`. Les couches supérieures ne voient donc jamais de code d'erreur PostgreSQL.

Côté client

Le client distingue trois familles de situations, et son comportement change dans chaque cas :

Situation	Comportement
Erreur de saisie (<code>400</code> , <code>409</code>)	Le formulaire est réaffiché, avec les valeurs déjà saisies et les messages sous les champs concernés. Le statut de l'API est conservé : un formulaire réaffiché après un refus n'est pas un succès
Jeton refusé (<code>401</code>)	La session est effacée et l'utilisateur est redirigé vers la connexion
API injoignable ou en défaut	Page d'erreur, mais la session est conservée

Pourquoi cette dernière distinction compte

Déconnecter quelqu'un parce qu'un service distant est momentanément absent l'obligerait à ressaisir ses identifiants sans raison : son jeton est valide, c'est l'API qui est en cause. Le comportement a été vérifié en arrêtant réellement le serveur : la page d'erreur s'affiche, le cookie survit, et la navigation reprend sans reconnexion au redémarrage de l'API.

6.4 Validation des données

La validation est déclarée par des tags `binding` sur les structures de requête, au plus près des champs qu'elle contraint :

```
type registerRequest struct {
    Email    string `json:"email"    binding:"required,email,max=255"`
    Password string `json:"password" binding:"required,min=8,max=72"`
    Name     string `json:"name"     binding:"required,min=2,max=100"`
}
```

Les erreurs sont retournées **champ par champ**, sous une forme directement exploitable par un formulaire :

```
{
  "error": "données invalides",
  "fields": {
    "email": "adresse email invalide",
    "password": "ce champ doit contenir au moins 8 caractères"
  }
}
```

Une correspondance est enregistrée au démarrage (`ConfigureValidation`) pour que les clés de `fields` portent les noms **JSON** des champs et non leurs noms Go. Sans elle, le serveur répondrait « SpaceID » là où le client a envoyé « space_id », et le formulaire ne saurait pas à quel champ rattacher le message.

Un dernier détail distingue les deux formulaires de note : l'état est facultatif à la création — une note nouvelle est « non faite » par défaut — mais **obligatoire** à la modification. Un `PUT` remplace l'intégralité de la ressource ; accepter un état absent le réinitialiserait silencieusement.

6.5 Persistance

L'accès se fait par `database/sql` avec le pilote `pgx`, en requêtes préparées paramétrées (`$1`, `$2` ...). Aucune requête n'est construite par concaténation de chaînes : l'injection SQL est écartée par construction, non par filtrage des entrées.

Le pool de connexions est explicitement borné (25 connexions ouvertes au maximum, 5 inactives, durée de vie de 5 minutes). Sans ces réglages, Go autorise un nombre illimité de connexions et peut saturer PostgreSQL sous charge.

Deux détails d'implémentation méritent d'être signalés :

- **Le nombre de notes par espace est calculé par jointure** dans la requête de listage, et non par une requête par espace. C'est ce qui évite le problème dit « N+1 ».
- `rows.Err()` **est vérifié après chaque parcours**. Sans cette vérification, une erreur survenant en cours d'itération passerait pour une liste simplement terminée, et le client recevrait une liste tronquée présentée

comme complète.

Les suppressions inspectent `RowsAffected` : un `DELETE` qui ne correspond à aucune ligne ne produit pas d'erreur SQL. Sans ce contrôle, supprimer une ressource inexistante ou étrangère répondrait « succès ».

6.6 Gestion des états de note

L'état est un type nommé plutôt qu'une simple chaîne, ce qui rend les valeurs autorisées visibles dans le code et centralise leur validation :

```
type NoteStatus string

const (
    StatusTodo      NoteStatus = "todo"
    StatusInProgress NoteStatus = "in_progress"
    StatusDone      NoteStatus = "done"
)
```

La même liste est contrainte à trois endroits, chacun couvrant ce que les autres ne peuvent pas : un `CHECK` en base (protège des écritures hors application), un tag `oneof` sur les requêtes (donne un message d'erreur exploitable), et la méthode `IsValid()` dans le service (protège si le service est appelé depuis un autre point d'entrée).

Les libellés français sont portés par une méthode `Label()` appelée directement par les gabarits. La liste des états proposés dans les formulaires vient également du code Go : ajouter un état ne demandera pas de retoucher le HTML.

6.7 Contraintes propres au client HTML

Les formulaires ne savent faire que GET et POST

Un navigateur ne peut pas émettre `PUT` ni `DELETE` depuis un formulaire. Le client expose donc des routes `POST` explicites, et c'est le client API qui les traduit :

Action	Route du client	Appel vers l'API
Modifier un espace	<code>POST /spaces/{id}/edit</code>	<code>PUT /api/spaces/{id}</code>
Supprimer un espace	<code>POST /spaces/{id}/delete</code>	<code>DELETE /api/spaces/{id}</code>
Modifier une note	<code>POST /notes/{id}/edit</code>	<code>PUT /api/notes/{id}</code>
Supprimer une note	<code>POST /notes/{id}/delete</code>	<code>DELETE /api/notes/{id}</code>

L'API REST reste ainsi correcte et l'application fonctionne sans une ligne de JavaScript. L'alternative — un champ caché `_method` interprété par le serveur — aurait fait porter à l'API une contrainte propre au HTML.

Les suppressions sont en `POST` et non en `GET` pour une autre raison : un lien `GET` peut être déclenché par le préchargement d'un navigateur ou par une image distante. Une suppression partirait alors sans clic de l'utilisateur. La déconnexion suit la même règle.

Session dans un cookie HttpOnly

Faute de JavaScript pour garder un jeton en mémoire, le client le stocke dans un cookie dont chaque attribut répond à une menace précise :

Attribut	Ce qu'il empêche
HttpOnly	La lecture du jeton depuis JavaScript, donc son exfiltration en cas de faille XSS
SameSite=Lax	L'envoi du cookie lors de requêtes venues d'un autre site, donc les attaques CSRF sur les formulaires de modification et de suppression
Expires aligné sur le jeton	L'envoi de requêtes vouées à échouer : le navigateur oublie le cookie quand le jeton devient invalide

POST puis redirection

Toute action de modification répond par une redirection 303 See Other et non par une page. L'utilisateur peut ainsi actualiser sans rejouer l'action, et le bouton « retour » ne propose pas de renvoyer le formulaire. Le 303 est préféré au 302 car il impose au navigateur de suivre la redirection en GET.

Le message de confirmation doit alors survivre à la redirection : il est porté par un cookie de courte durée, lu et **immédiatement effacé** au rendu suivant. Un message de confirmation qui resterait affiché après un rechargement serait trompeur.

Échappement des contenus

html/template échappe selon le contexte, sans intervention. Une note intitulée `<script>alert(1)</script>` a été créée pour le vérifier :

Contexte de rendu	Résultat
Texte HTML	<code>&lt;script&gt;</code>
Attribut <code>value=""</code>	<code>&lt;script&gt;</code>
Chaîne JavaScript (<code>onsubmit</code>)	<code>\u003cscript\u003e</code>

Le même contenu est encodé de **trois manières différentes** selon l'endroit où il apparaît. Aucune balise n'est produite dans le document, et la charge reste du texte inerte.

Gabarits compilés au démarrage

Les gabarits sont compilés une seule fois au lancement : une erreur de syntaxe empêche le démarrage plutôt que d'apparaître au hasard d'une navigation. Le rendu passe en outre par un tampon mémoire avant d'être écrit dans la réponse. Sans ce détour, un gabarit qui échoue à mi-parcours aurait déjà envoyé au navigateur une page tronquée, impossible à remplacer par une page d'erreur puisque le statut serait déjà parti.

7. Stratégie de tests

Les tests ont été écrits là où ils apportent une garantie, et non pour atteindre un taux de couverture. Ils se répartissent en trois natures.

Tests unitaires — serveur

Ils couvrent ce qui est purement algorithmique : hachage et vérification des mots de passe, génération et validation des jetons, validité des états de note. Les cas d'attaque y figurent explicitement : jeton signé avec un autre secret, jeton expiré, signature altérée, jeton `alg=none`.

Tests d'intégration — serveur

Ils s'exécutent **contre une véritable base PostgreSQL**. C'est un choix assumé : ce que l'on veut vérifier ici est le comportement du SQL lui-même — filtrage par utilisateur dans les clauses `WHERE`, contraintes d'unicité, suppressions en cascade. Les simuler avec un faux repository ne prouverait rien de ce qui compte.

Deux précautions les rendent utilisables au quotidien :

- Sans la variable `TEST_DATABASE_URL`, ils sont **ignorés** et non en échec : `go test ./...` reste exécutable sans base disponible.
- Chaque test crée ses propres utilisateurs jetables et les supprime à la fin. La cascade du schéma emporte leurs espaces et leurs notes. Les données de démonstration ne sont jamais modifiées, et la suite peut être relancée indéfiniment — ce qui a été vérifié en comparant l'état de la base avant et après.

Tests de bout en bout — client

Ils montent l'application complète — routeur, handlers et **gabarits réels** — devant une fausse API construite avec `httptest`. Ils vérifient donc ce qu'un navigateur recevrait vraiment : codes de statut, redirections, cookies et HTML produit. Ni serveur ni base de données ne sont nécessaires pour les lancer.

Trois d'entre eux portent l'essentiel de la valeur :

Test	Ce qu'il verrouille
<code>TestClientUsesExpectedMethodAndPath</code>	Qu'un <code>POST /spaces/7/edit</code> venu du navigateur part bien en <code>PUT /api/spaces/7</code>
<code>TestAPIFailureDoesNotDestroySession</code>	Qu'une API en défaut n'entraîne pas de déconnexion — le test jumeau <code>TestRejectedTokenClearsSessionAndRedirects</code> vérifiant qu'un jeton refusé, lui, déconnecte bien
<code>TestNoteTitleIsEscaped</code>	Que l'échappement est appliqué, et différemment selon le contexte

Vérification manuelle

Chaque fonctionnalité a en outre été éprouvée par des requêtes réelles contre le serveur puis le client en fonctionnement : cas nominaux, cas limites (identifiants non numériques ou négatifs, JSON malformé, champs

hors bornes) et tentatives d'accès croisé entre deux comptes. Les résultats sont repris dans les sections correspondantes.

8. Méthode de travail et versionnement

Le projet a été construit couche par couche, chaque étape sur sa propre branche mergée dans `main` avec un commit de fusion. L'historique conserve ainsi le découpage du travail et se lit comme la progression du projet.

Serveur	Client
chore: project structure	chore: project structure
feat: database schema	feat: typed HTTP API client
feat: authentication	feat: template rendering
feat: space management	feat: session and authentication
feat: note management	feat: space pages
chore: dependency upgrade	feat: note pages

Chaque commit a été vérifié avant d'être enregistré : `gofmt`, `go build`, `go vet` et la suite de tests. Chaque commit de l'historique compile donc de façon autonome.

Un fichier `.gitattributes` normalise les fins de ligne en LF dans les deux dépôts. Le développement s'est fait sous Windows : sans cette règle, les fichiers auraient été versionnés avec des CRLF, ce qui empêche notamment `make` et les scripts shell de fonctionner sous Linux.

Aucun secret n'est versionné. Les deux dépôts fournissent un `.env.example` et ignorent `.env`.

Un exemple de correction issue de cette méthode

La mise à jour des dépendances en fin de projet a fait apparaître un test en échec, `TestParseRejectsTamperedToken`. Le diagnostic mérite d'être rapporté, car c'est le test qui était faux et non la bibliothèque.

Le test altérait le **dernier** caractère de la signature. Or une signature HS256 fait 32 octets, encodés en 43 caractères `base64url` qui en portent 258 bits : les deux derniers bits sont du remplissage. Quatre caractères finaux différents décodent donc vers les mêmes octets. L'ancienne version de la bibliothèque rejetait ce `base64` non canonique — le test passait pour cette raison, et non pour celle qu'il annonçait. La nouvelle l'accepte, révélant le défaut.

Le test altère désormais le **premier** caractère de la signature, qui porte des bits significatifs, et vérifie explicitement que le jeton modifié diffère de l'original. Le comportement de sécurité n'a jamais été en cause.

9. Limites, simplifications et compromis

Les points suivants sont des choix conscients, pas des oublis. Ils sont classés du plus au moins structurant.

9.1 Sécurité

Pas de jeton de rafraîchissement. Le jeton d'accès vit 24 heures ; passé ce délai, l'utilisateur se reconnecte. Un couple accès court / rafraîchissement long serait la solution complète, au prix d'un stockage de jetons côté serveur. Hors périmètre du sujet.

Pas de révocation possible. Une API sans état ne peut pas invalider un jeton déjà émis. La déconnexion efface le cookie du navigateur, mais le jeton reste techniquement valide jusqu'à son expiration. Une liste de révocation serait nécessaire — et ferait perdre l'absence d'état.

Secure absent du cookie de session. Le développement se fait en HTTP. En production derrière HTTPS, cet attribut est indispensable pour empêcher l'envoi du cookie en clair.

Pas de jeton anti-CSRF dédié. La protection repose sur `SameSite=Lax`, qui couvre les navigateurs actuels. Un jeton par formulaire serait la défense complète.

Pas de limitation du nombre de tentatives de connexion. Rien n'empêche d'essayer des mots de passe en série. bcrypt ralentit chaque tentative, ce qui atténue le problème sans le résoudre. Une limitation par adresse IP ou par compte serait attendue en production.

Identifiants séquentiels. Les `BIGSERIAL` permettent de deviner les identifiants voisins. L'impact est nul ici puisque toute ressource étrangère répond `404`, mais des UUID empêcheraient même la tentative.

Réponse de connexion à durée variable. Un email inconnu renvoie sans exécuter bcrypt, un email connu l'exécute. L'écart de temps permet théoriquement de deviner si une adresse est enregistrée. Un calcul bcrypt factice dans le cas « inconnu » supprimerait cet écart.

9.2 Fonctionnalités

Pas de pagination. Les listes d'espaces et de notes sont retournées en entier. Acceptable à l'échelle d'un utilisateur, à revoir au-delà de quelques centaines de notes par espace.

Une note ne peut pas changer d'espace. `space_id` n'est pas modifiable. Le sujet ne le demande pas, et l'autoriser impliquerait de vérifier l'appartenance de l'espace de destination.

Pas de recherche ni de filtrage par état. Les notes sont triées de la plus récente à la plus ancienne, sans autre option.

Aucun partage entre utilisateurs. Le modèle associe un espace à exactement un utilisateur, conformément à l'énoncé. C'est ce qui rend le contrôle d'accès aussi simple à garantir.

9.3 Exploitation

Migrations jouées au premier démarrage du conteneur. Les fichiers de `migrations/` ne sont exécutés qu'à la création du volume PostgreSQL. Modifier le schéma impose de recréer la base (`make db-reset`). Un outil de migration versionnée serait nécessaire dès qu'il faut faire évoluer un schéma contenant des données réelles.

Le serveur n'est pas conteneurisé. Seule la base l'est. C'est un choix de confort en développement — recompiler est instantané — mais un `Dockerfile` serait attendu pour un déploiement.

Journalisation textuelle. Le paquet `log` standard est utilisé. Des journaux structurés (`log/slog`) avec un identifiant de corrélation par requête seraient préférables pour exploiter la production.

Pas d'intégration continue. Les vérifications (`gofmt`, `vet`, `tests`) ont été lancées à la main avant chaque commit. Un workflow GitHub Actions les rendrait systématiques.

Le profil est relu à chaque requête du client. Chaque page déclenche un appel à `/api/me`. C'est un aller-retour supplémentaire, assumé au profit de la simplicité : le jeton est ainsi toujours vérifié et le nom affiché toujours à jour.

10. Mise en route

Prérequis : **Go 1.25** ou supérieur, **Docker** et Docker Compose.

```
# 1. Serveur
git clone https://github.com/KxroTM/test-technique-ynov.git
cd test-technique-ynov
cp .env.example .env
docker compose up -d          # PostgreSQL + schéma + données de démonstration
go run .                      # API sur http://localhost:8080

# 2. Client, dans un autre terminal
git clone https://github.com/KxroTM/test-technique-ynov-client.git
cd test-technique-ynov-client
cp .env.example .env
go run ./cmd/web              # Application sur http://localhost:3000
```

Les valeurs par défaut des deux `.env.example` sont fonctionnelles en local : aucune modification n'est nécessaire.

Comptes de démonstration

Email	Mot de passe	Contenu
<code>alice@example.com</code>	<code>password123</code>	2 espaces (« Devoirs », « Jobs »), 3 notes
<code>bob@example.com</code>	<code>password123</code>	1 espace (« Personnel »), 1 note

Le second compte n'est pas là pour faire nombre. Connectez-vous en tant qu'Alice, relevez l'identifiant d'un de ses espaces, puis connectez-vous en tant que Bob et saisissez l'URL de cet espace : la page « introuvable » s'affiche. C'est la démonstration la plus directe du cloisonnement décrit en section 6.2.

Lancer les tests

```
# Serveur – tests unitaires
go test ./...

# Serveur – tests d'intégration (base démarrée requise)
TEST_DATABASE_URL="postgres://notes_user:notes_password@localhost:5432/notes_db?sslmode=disable" \
go test ./internal/repository -v

# Client – aucune dépendance requise
go test ./...
```

Un `Makefile` regroupe ces commandes dans les deux dépôts (`make help` en donne la liste).